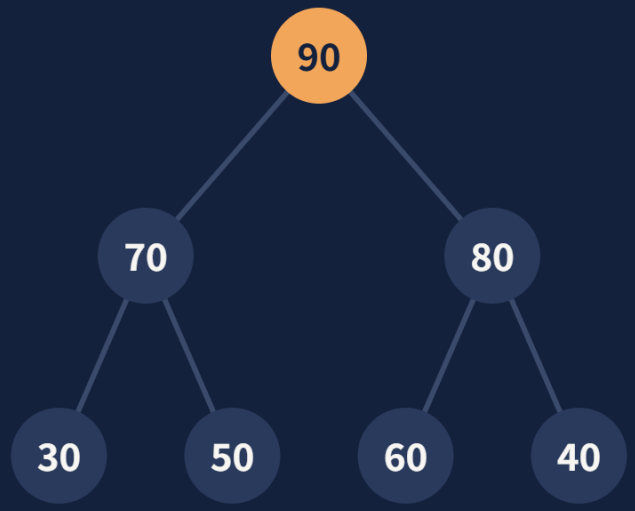


CS 스터디 · 4회차 · Data Structure

Heap & Priority Queue
Hash Table

우선순위대로 꺼내는 법, 키로 바로 찾는 법

발표 · 권혜강

PRIORITY QUEUE · 01

우선순위 큐: 순서가 아니라 우선순위로

큐 (Queue)

먼저 들어온 것이 먼저 나간다 (FIFO)

넣은 순서 3 → 1 → 2
꺼낸 순서 3 → 1 → 2

우선순위 큐 (Priority Queue)

우선순위가 가장 높은 것이 먼저 나간다

넣은 순서 3 → 1 → 2
꺼낸 순서 1 → 2 → 3 (작은 값 우선)

우선순위 큐는 **"무엇을 하는가"**를 정한 추상 자료형(ADT), 힙은 그것을 **"어떻게 구현하는가"**에 해당하는 자료구조

PRIORITY QUEUE · 02

왜 하필 힙으로 구현할까?

구현 방법	삽입	삭제 (꺼내기)	조회 (peek)
정렬 안 된 배열	O(1)	O(n)	O(n)
정렬된 배열	O(n)	O(1)	O(1)
정렬된 연결 리스트	O(n)	O(1)	O(1)
힙 (Heap)	O(log n)	O(log n)	O(1)

배열·리스트는 삽입과 삭제 중 **하나가 O(n)**. 힙은 **둘 다 O(log n)**이라 삽입·삭제가 섞여 반복될 때 가장 유리하다.

HEAP · 01

힙 = 완전 이진 트리 + 힙 속성

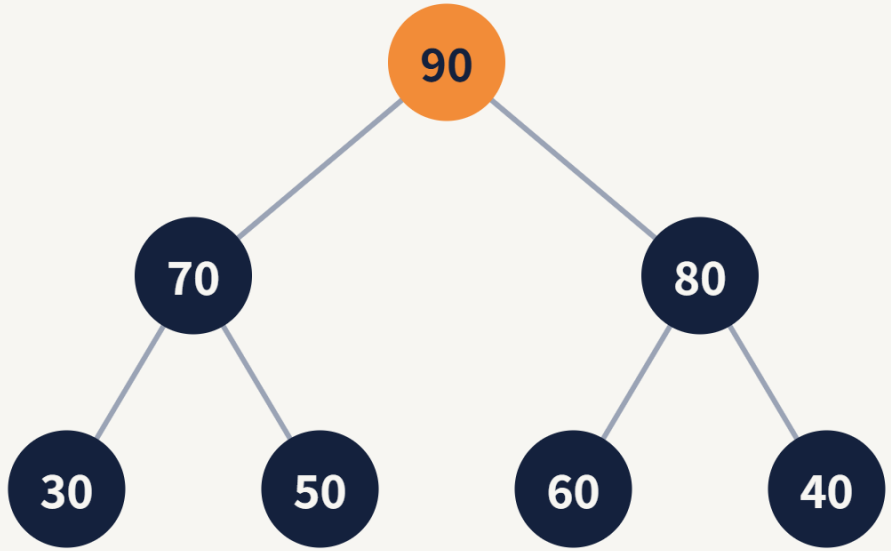
① 완전 이진 트리

위에서 아래, 왼쪽에서 오른쪽으로 빈틈없이 채운다 → 배열에 그대로 저장 가능

② 힙 속성

최대 힙: 부모 ≥ 자식 → 루트가 최댓값
최소 힙: 부모 ≤ 자식 → 루트가 최솟값

주의: 형제끼리는 순서가 없다. 힙은 '정렬된' 구조가 아니다 (반정렬).



최대 힙 예시: 루트(90)가 최댓값

HEAP · 02

트리지만 배열에 저장한다

[0]	[1]	[2]	[3]	[4]	[5]	[6]
90	70	80	30	50	60	40

부모

`(i - 1) // 2`

왼쪽 자식

`2i + 1`

오른쪽 자식

`2i + 2`

예) [1]의 70 → 자식은 [3] 30, [4] 50 · 1번 인덱스부터 쓰면 부모 i // 2, 자식 2i, 2i + 1

삽입 (push): 맨 끝에 넣고 위로 올린다

1. 배열의 **맨 끝**에 새 값을 추가한다
2. **부모와 비교**해서 힙 속성이 깨졌으면 교환
3. 루트에 닿거나 부모가 더 크면 멈춤 (sift-up)

비교 횟수 ≤ 트리 높이 → O(log n)

예) 최대 힙에 85 삽입

```
[90, 70, 80, 30, 50, 60, 40, 85]
85 > 부모 30 → 교환
[90, 70, 80, 85, 50, 60, 40, 30]
85 > 부모 70 → 교환
[90, 85, 80, 70, 50, 60, 40, 30]
85 < 부모 90 → 멈춤, 완료
```

삭제 (pop): 루트를 꺼내고 아래로 내린다

1. **루트**(최댓값)를 꺼낸다
2. **마지막 원소**를 루트 자리로 옮긴다
3. **더 큰 자식**과 교환하며 내려간다 (sift-down)

역시 높이만큼 → O(log n)

예) 최대 힙에서 pop → 90

```
[40, 70, 80, 30, 50, 60]
마지막 40을 루트로 · 큰 자식 80과 교환
[80, 70, 40, 30, 50, 60]
40 < 자식 60 → 교환
[80, 70, 60, 30, 50, 40]
자식 없음 → 완료
```

Heapify와 힙 정렬

HEAPIFY · 배열 → 힙

O(n)

마지막 내부 노드 (n // 2 - 1)부터 루트까지 거꾸로 sift-down. 하나씩 n번 push하면 O(n log n)이지만, 한 번에 만들면 O(n).

HEAP SORT · 힙 정렬

O(n log n)

힙을 만든 뒤 루트를 n번 꺼낸다. 최선·평균·최악 모두 O(n log n), 추가 메모리 O(1), **불안정 정렬**.

PART 2 · HASH TABLE

키로 값을 바로 찾는다

배열의 '인덱스로 O(1) 접근'을 아무 키에나 쓸 수 있게 만든 구조
→ 검색·삽입·삭제 **평균 O(1)**

Python dict · Java HashMap · JS Map · DB 해시 인덱스 · 캐시

HASH TABLE · 01

동작 원리: 키 → 해시 함수 → 인덱스



같은 키는 **항상 같은 인덱스**로 간다 → 찾을 때도 해시 계산 한 번이면 위치를 안다. 버킷(bucket) 배열에 **(키, 값)** 쌍을 저장.

HASH TABLE · 03

해시 충돌 (Collision)

서로 다른 키가
같은 인덱스로 가는 것

비둘기집 원리: 키의 가짓수는 사실상 무한, 버킷 수는 유한 → 충돌은 **피할 수 없다**.
핵심은 충돌을 **어떻게 처리하느냐**.

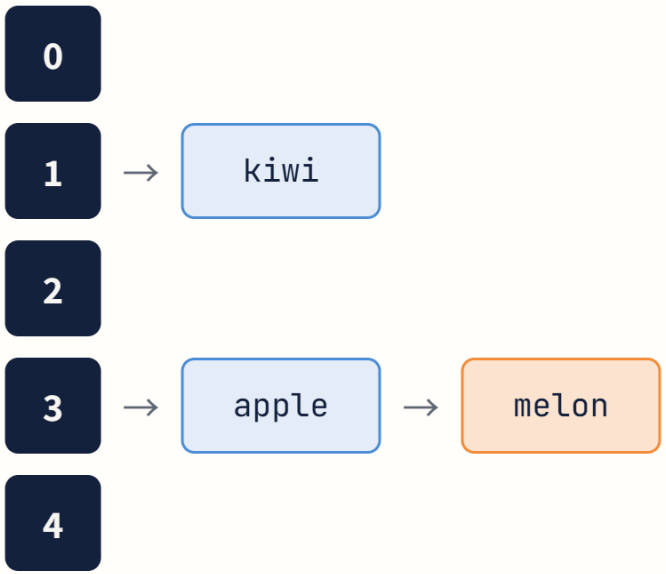
```
h("apple") % 8 = 3
h("melon") % 8 = 3
```

⚠ **충돌! 3번 버킷에 누구를 둘까?**

해결 1 · 체이닝 (Separate Chaining)

해결 2 · 개방 주소법 (Open Addressing)

체이닝: 같은 버킷에 줄줄이 연결



충돌한 원소를 버킷의 **연결 리스트**에 이어 붙인다. 찾을 때는 그 리스트만 훑는다.

장점

구현이 단순하고, 원소가 버킷 수보다 많아도 동작

단점

한 버킷에 몰리면 O(n), 노드 포인터만큼 메모리 추가, 캐시 효율 낮음

Java 8+ HashMap

한 버킷이 8개 이상 + 테이블 64칸 이상이면 리스트 → **레드-블랙 트리**로 전환, 최악 O(log n)

개방 주소법: 테이블 안의 빈칸을 찾는다



melon: 3번이 차 있음 → 다음 칸 4번에 저장 (선형 탐사)

선형 탐사

`h(k) + i`

구현이 쉽지만 연속으로 뭉치는 **1차 군집** 발생

이차 탐사

$h(k) + i^2$

간격을 넓혀 군집 완화, 같은 시작점끼리는 **2차 군집**

이중 해싱

`h1(k) + i·h2(k)`

두 번째 해시로 간격 결정, 군집이 가장 적음

삭제 주의: 그냥 비우면 탐색이 끊긴다 → '삭제됨(tombstone)' 표시 · **Python dict**가 개방 주소법을 사용

HASH TABLE · 07

로드 팩터와 리해싱

$$\alpha = n / m$$

n = 저장된 원소 수
m = 버킷 수
α가 커질수록 충돌이 늘어난다

예) Java HashMap

기본 용량	로드 팩터	임계값
16	0.75	12

원소가 12개를 넘으면 → 용량 **2배(32)**로 늘리고 모든 원소를 새 위치로 다시 배치 (**rehashing**)

리해싱 한 번은 O(n)이지만 드물게 일어나므로 삽입은 **분할 상환(amortized) O(1)** · 크기를 알면 초기 용량을 미리 지정하자

INTERVIEW · HEAP

면접 예상 질문: 힙 & 우선순위 큐

Q. 힙과 이진 탐색 트리의 차이는?

BST는 왼쪽 < 부모 < 오른쪽으로 전체가 정렬, 검색 O(log n). 힙은 부모-자식 관계만 → 최댓/최솟값 O(1), 임의 검색은 O(n).

Q. 힙을 배열로 구현하는 이유는?

완전 이진 트리라 빈칸이 없어 인덱스 계산만으로 부모·자식 접근. 포인터가 필요 없고 캐시 효율이 좋다.

Q. 우선순위 큐를 힙으로 만드는 이유는?

배열·리스트는 삽입과 삭제 중 하나가 O(n). 힙은 둘 다 O(log n)이라 반복 작업에 유리하다.

Q. Heapify가 O(n)인 이유는?

노드 대부분이 아래쪽에 있어 내려갈 거리가 짧다. 높이별 (노드 수 × 이동 거리)의 합이 O(n)에 수렴.

INTERVIEW · HASH TABLE

면접 예상 질문: 해시 테이블

Q. 해시 충돌이란? 해결 방법은?

서로 다른 키가 같은 인덱스로 가는 것. 버킷에 리스트로 잇는 체이닝, 다른 빈칸을 찾는 개방 주소법.

Q. 최악의 시간 복잡도와 그 원인은?

O(n). 키가 한 버킷에 몰릴 때. Java 8 HashMap은 트리 변환으로 O(log n)까지 완화.

Q. 로드 팩터와 리해싱이란?

α = 원소 수 / 버킷 수. 임계값(Java 0.75)을 넘으면 테이블을 2배로 키우고 모든 원소를 다시 배치.

Q. HashMap과 TreeMap의 차이는?

HashMap은 순서 없이 평균 O(1). TreeMap은 레드-블랙 트리로 키를 정렬 유지, O(log n).

SUMMARY

힙

최댓/최솟값을 O(1)에 보고, O(log n)에 넣고 뺀다

우선순위 큐

ADT. 보통 힙으로 구현한다

해시 테이블

평균 O(1). 충돌 처리와 로드 팩터가 성능을 결정